

```
In [19]: import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
```

```
In [21]: data = pd.read_csv("loan_sanction_test.csv")
data.head()
```

```
Out[21]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	Applica
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

```
In [23]: print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               367 non-null   object
1   Gender                356 non-null   object
2   Married               367 non-null   object
3   Dependents            357 non-null   object
4   Education             367 non-null   object
5   Self_Employed         344 non-null   object
6   ApplicantIncome       367 non-null   int64
7   CoapplicantIncome     367 non-null   int64
8   LoanAmount            362 non-null   float64
9   Loan_Amount_Term      361 non-null   float64
10  Credit_History        338 non-null   float64
11  Property_Area         367 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
None
```

```
In [25]: print(data.describe())
```

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
count	367.000000	367.000000	362.000000	361.000000
mean	4805.599455	1569.577657	136.132597	342.537396
std	4910.685399	2334.232099	61.366652	65.156643
min	0.000000	0.000000	28.000000	6.000000
25%	2864.000000	0.000000	100.250000	360.000000
50%	3786.000000	1025.000000	125.000000	360.000000
75%	5060.000000	2430.500000	158.000000	360.000000
max	72529.000000	24000.000000	550.000000	480.000000

	Credit_History
count	338.000000
mean	0.825444
std	0.380150
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

```
In [27]: print(data.isnull().sum())
```

```
Loan_ID      0
Gender       11
Married      0
Dependents   10
Education    0
Self_Employed 23
ApplicantIncome 0
CoapplicantIncome 0
LoanAmount    5
Loan_Amount_Term 6
Credit_History 29
Property_Area 0
dtype: int64
```

```
In [29]: data.Gender.unique()
```

```
Out[29]: array(['Male', 'Female', nan], dtype=object)
```

```
In [31]: data.Gender = data.Gender.astype('category')
```

```
In [33]: data.Education = data.Education.astype('category')
```

```
In [35]: data.Married = data.Married.astype('category')
```

```
In [37]: data.Self_Employed = data.Self_Employed.astype('category')
```

```
In [39]: print(data.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               367 non-null   object
1   Gender                356 non-null   category
2   Married               367 non-null   category
3   Dependents            357 non-null   object
4   Education             367 non-null   category
5   Self_Employed         344 non-null   category
6   ApplicantIncome       367 non-null   int64
7   CoapplicantIncome     367 non-null   int64
8   LoanAmount            362 non-null   float64
9   Loan_Amount_Term      361 non-null   float64
10  Credit_History         338 non-null   float64
11  Property_Area          367 non-null   object
dtypes: category(4), float64(3), int64(2), object(3)
memory usage: 25.0+ KB
None

```

```
In [41]: data.Gender.mode()
```

```

Out[41]: 0    Male
         Name: Gender, dtype: category
         Categories (2, object): ['Female', 'Male']

```

```
In [43]: print(data.isnull().sum())
```

```

Loan_ID           0
Gender            11
Married           0
Dependents        10
Education          0
Self_Employed     23
ApplicantIncome    0
CoapplicantIncome  0
LoanAmount         5
Loan_Amount_Term   6
Credit_History    29
Property_Area      0
dtype: int64

```

```

In [45]: mode_gender = data['Gender'].mode() [0]
         data['Gender'].fillna(mode_gender, inplace =True)

```

C:\Users\saransh\AppData\Local\Temp\ipykernel_2020\3855755076.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

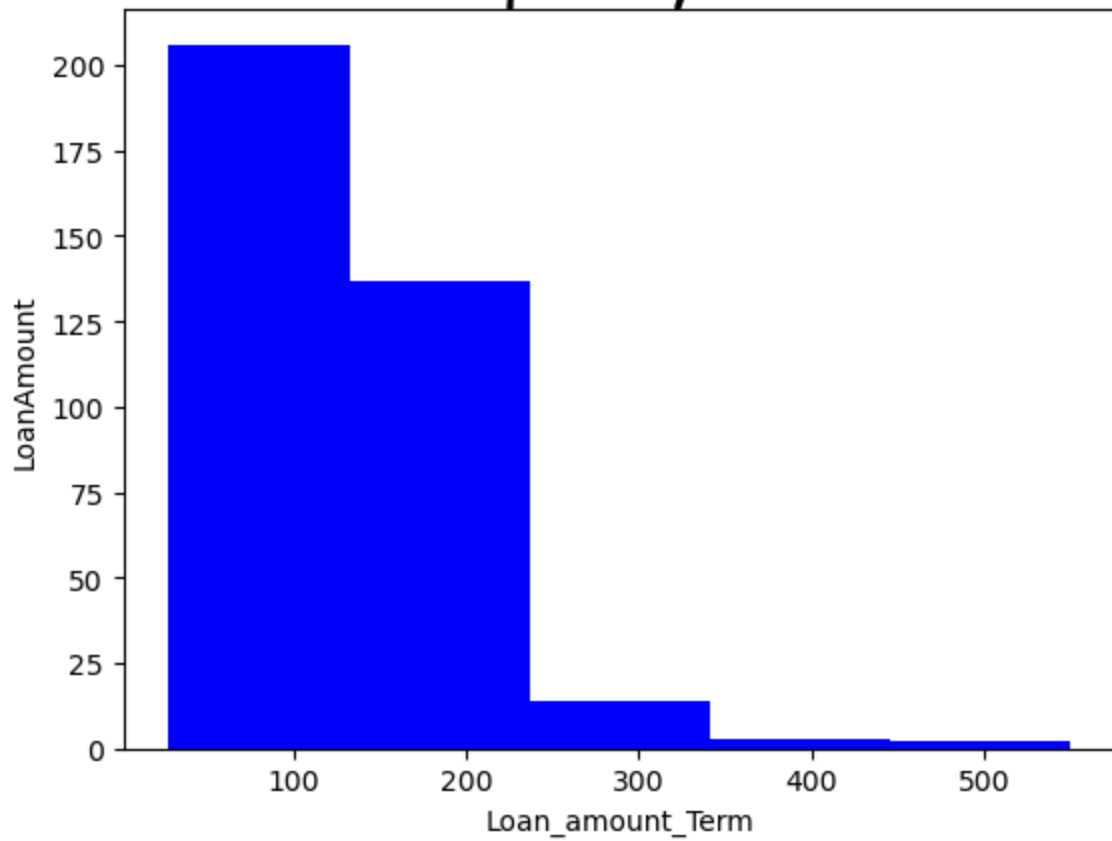
```
data['Gender'].fillna(mode_gender, inplace = True)
```

```
In [49]: print(data.isnull().sum())
```

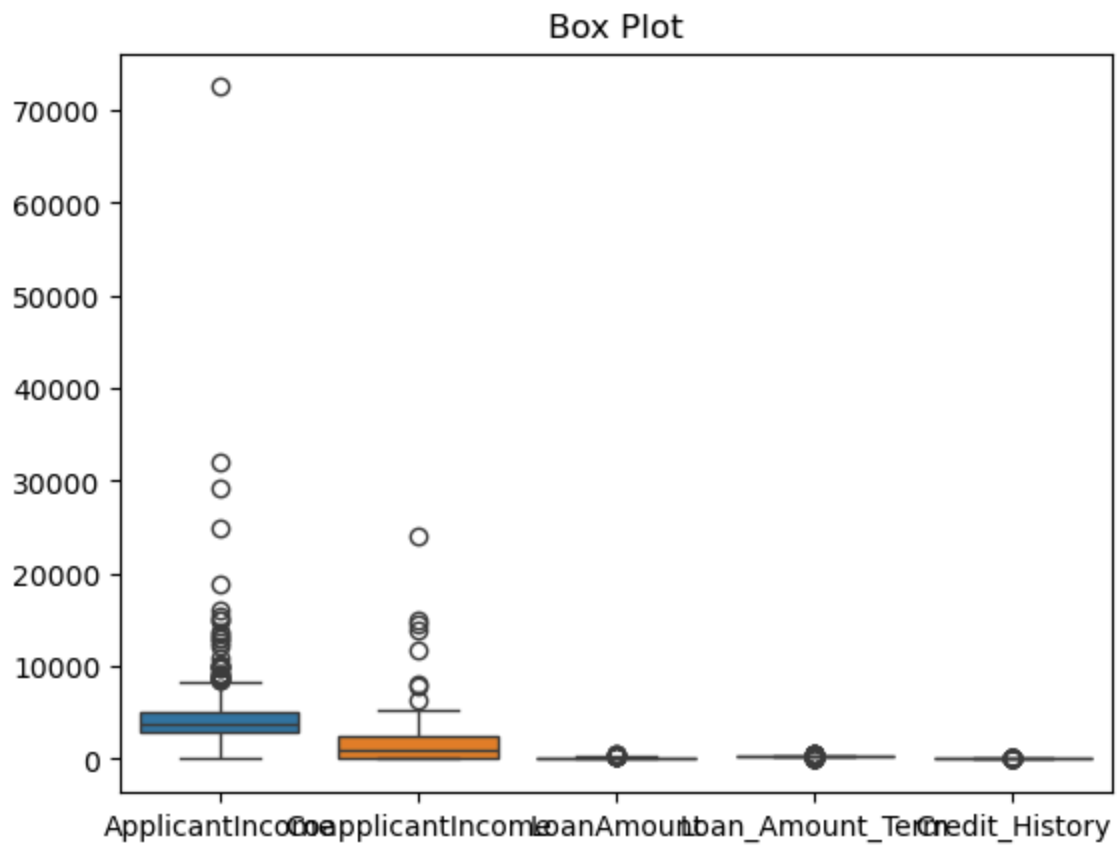
```
Loan_ID          0
Gender           0
Married          0
Dependents       10
Education         0
Self_Employed    23
ApplicantIncome  0
CoapplicantIncome 0
LoanAmount        5
Loan_Amount_Term  6
Credit_History   29
Property_Area     0
dtype: int64
```

```
In [51]: plt.hist(data.LoanAmount, bins = 5, color = "blue")
plt.title("frequency Distribution", loc = "right", font = "Comic Sans MS", size = 14)
plt.xlabel("Loan_amount_Term")
plt.ylabel("LoanAmount")
plt.show()
```

frequency Distribution



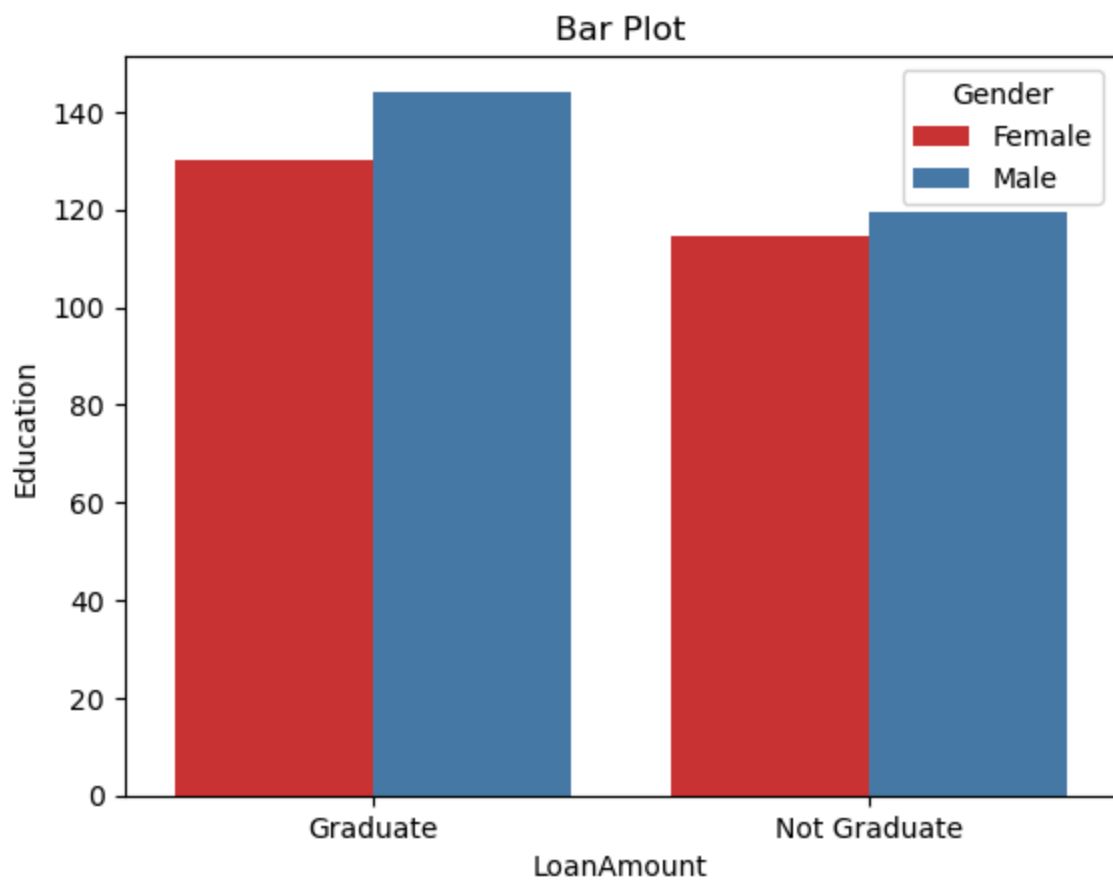
```
In [53]: sns. boxplot(data = data)
plt.title("Box Plot")
plt.show()
```



In []:

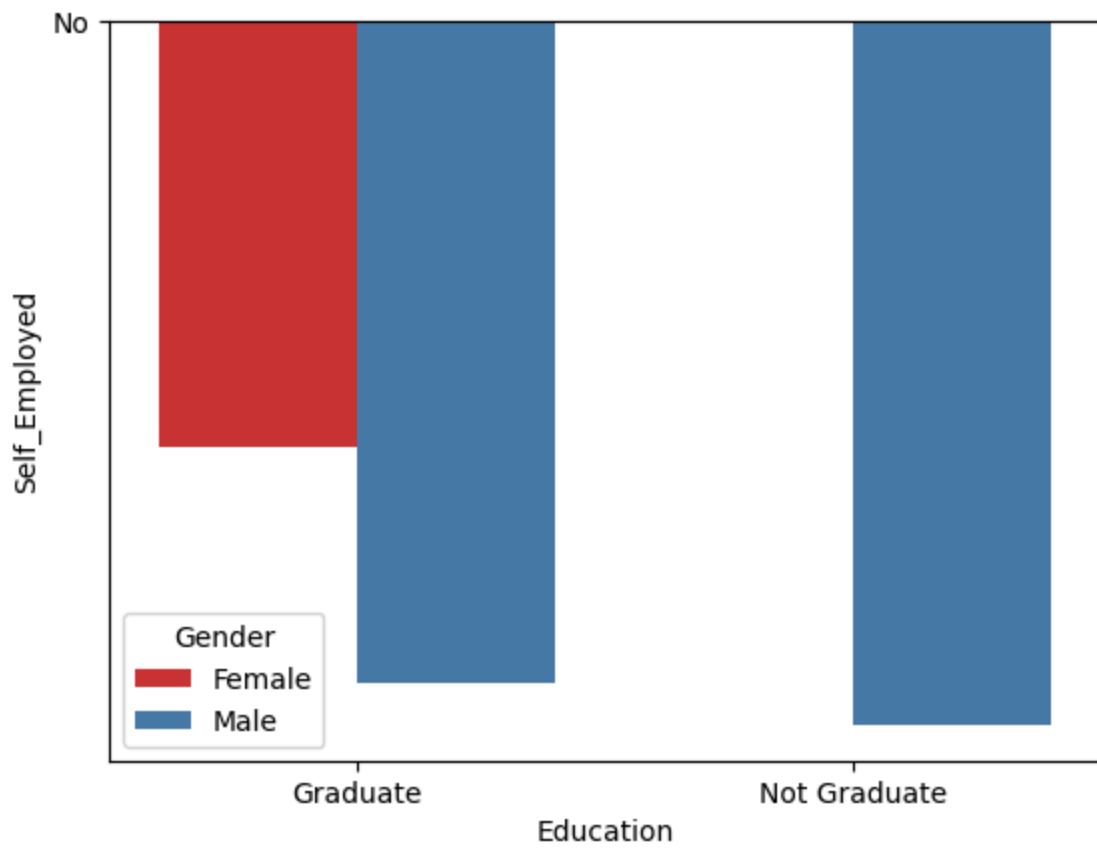
```
In [55]: sns.barplot(data , x = 'Education', y = 'LoanAmount', errorbar=('ci', False))
plt.xlabel('LoanAmount')
plt.ylabel('Education')
plt.title('Bar Plot')
```

Out[55]: Text(0.5, 1.0, 'Bar Plot')

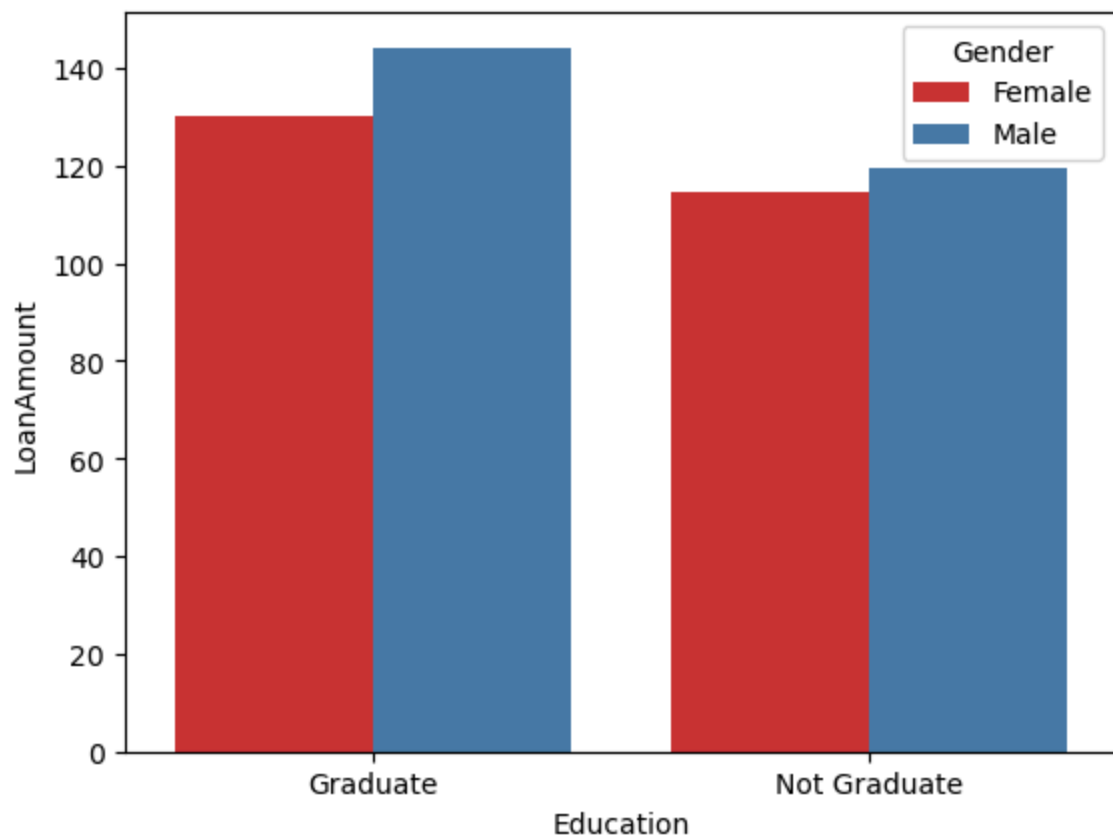


In []:

```
In [61]: sns.barplot(data = data , x = 'Education' , y = 'Self_Employed', errorbar
plt.show())
```



```
In [63]: sns.barplot(data=data, x='Education', y='LoanAmount', errorbar = ('ci' , False),  
plt.show())
```




```
In [65]: seg = data.groupby('Education')[['LoanAmount', 'ApplicantIncome']].sum().reset_index()
```

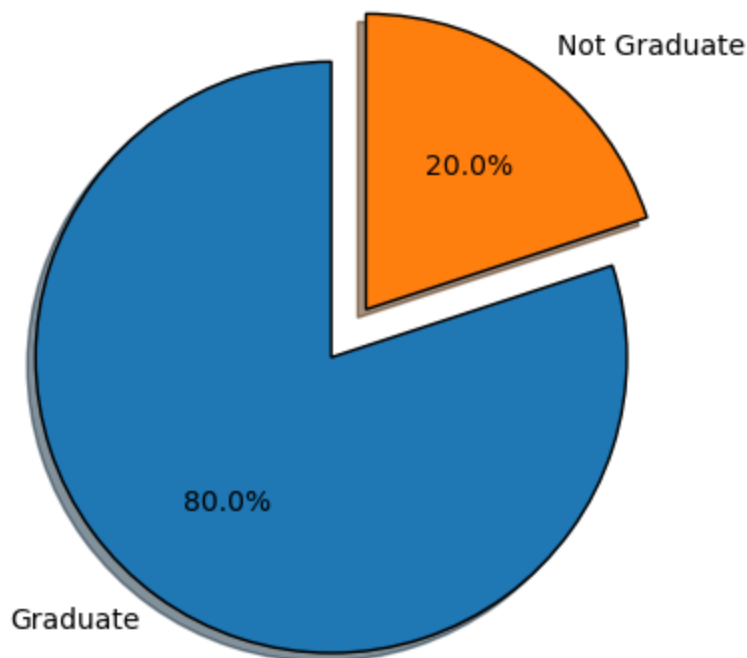
C:\Users\saransh\AppData\Local\Temp\ipykernel_2020\99350198.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
seg = data.groupby('Education')[['LoanAmount', 'ApplicantIncome']].sum().reset_index()
```

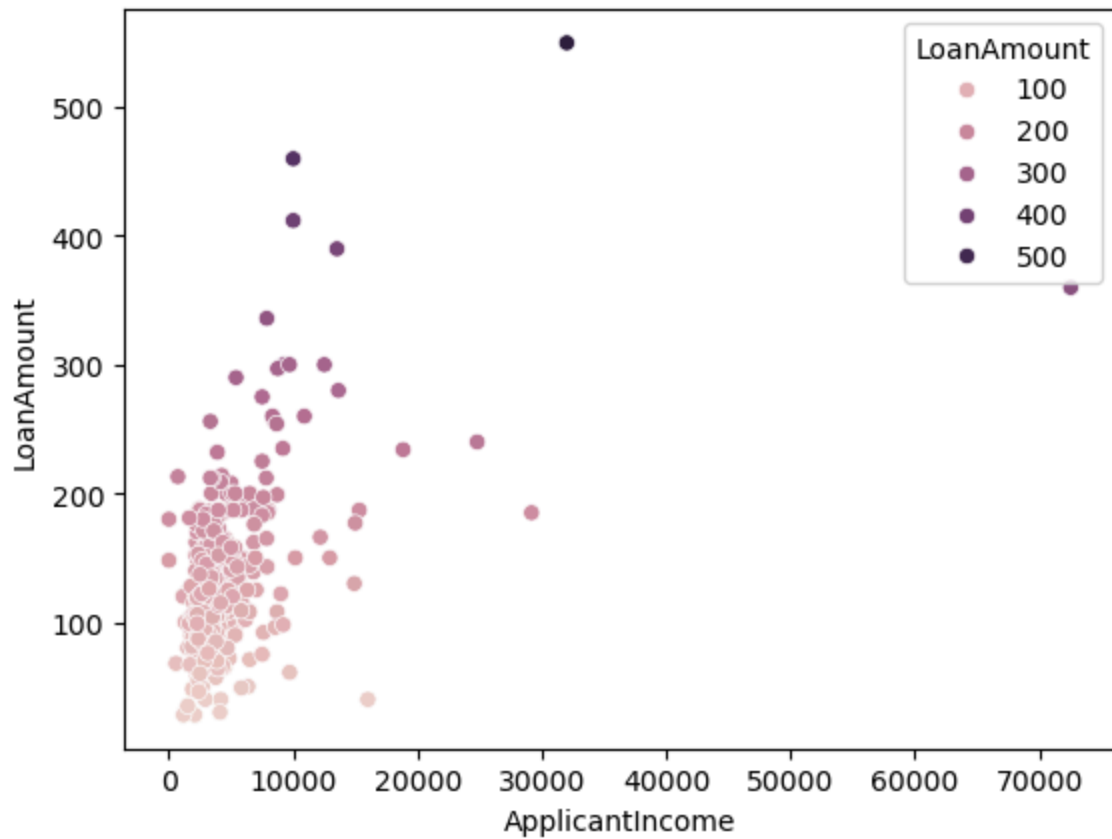
```
Out[65]:
```

	Education	LoanAmount	ApplicantIncome
0	Graduate	39439.0	1463094
1	Not Graduate	9841.0	300561

```
In [67]: plt.pie(x= seg.LoanAmount, labels = seg.Education, startangle = 90, shadow = True, autopct = f'%1.1f%%', explode = [0.1,0.1], wedgeprops = {"edgecolor": "black"}, plt.show())
```

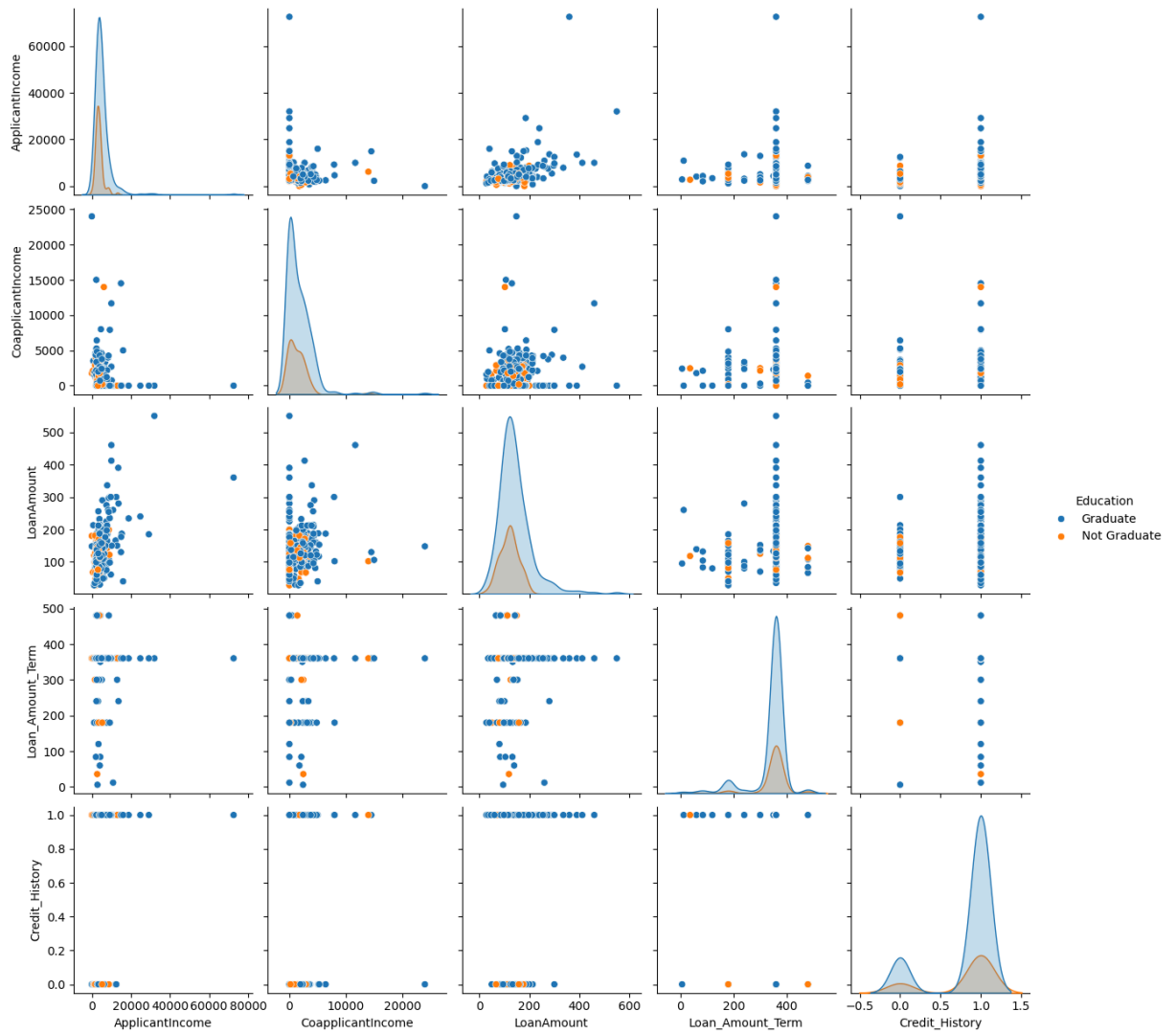


```
In [69]: sns.scatterplot(data = data , x = 'ApplicantIncome' , y = 'LoanAmount' , hue = 'Education', plt.show())
```

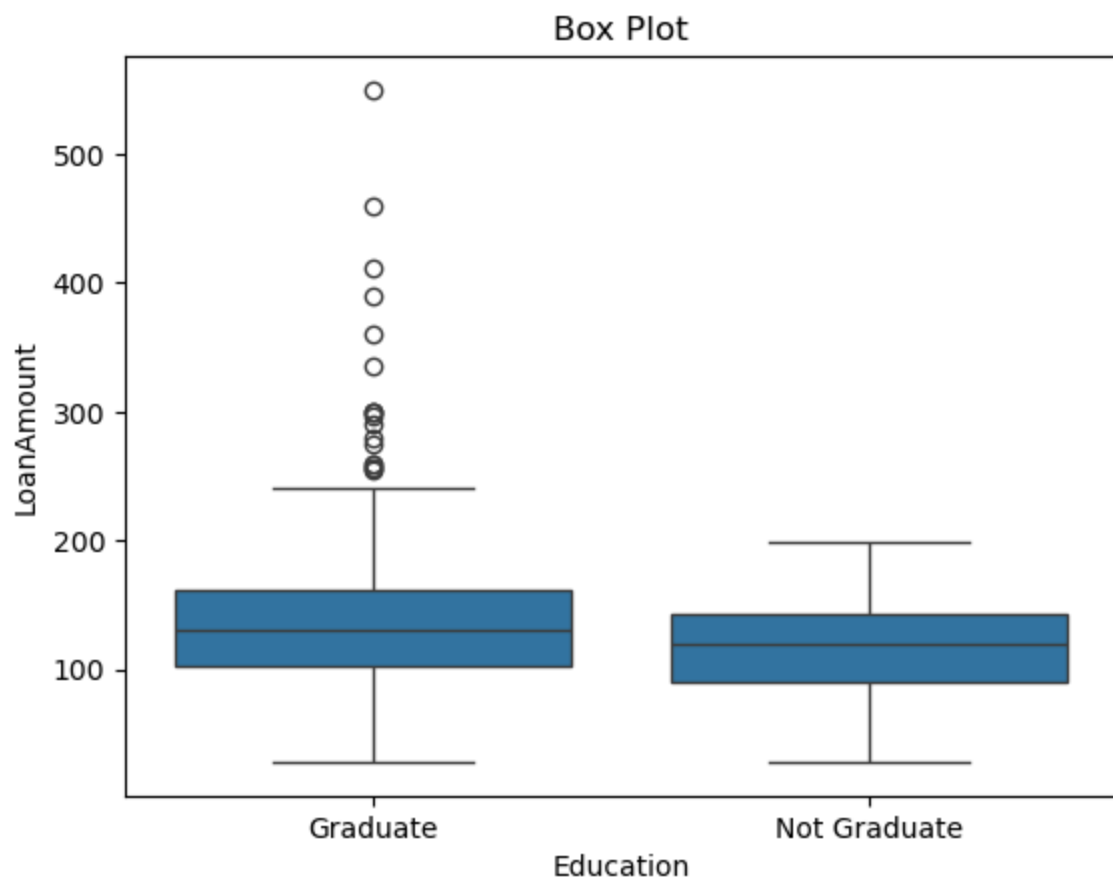


```
In [58]: sns.pairplot(data , hue = 'Education' , kind = 'scatter')  
plt.show
```

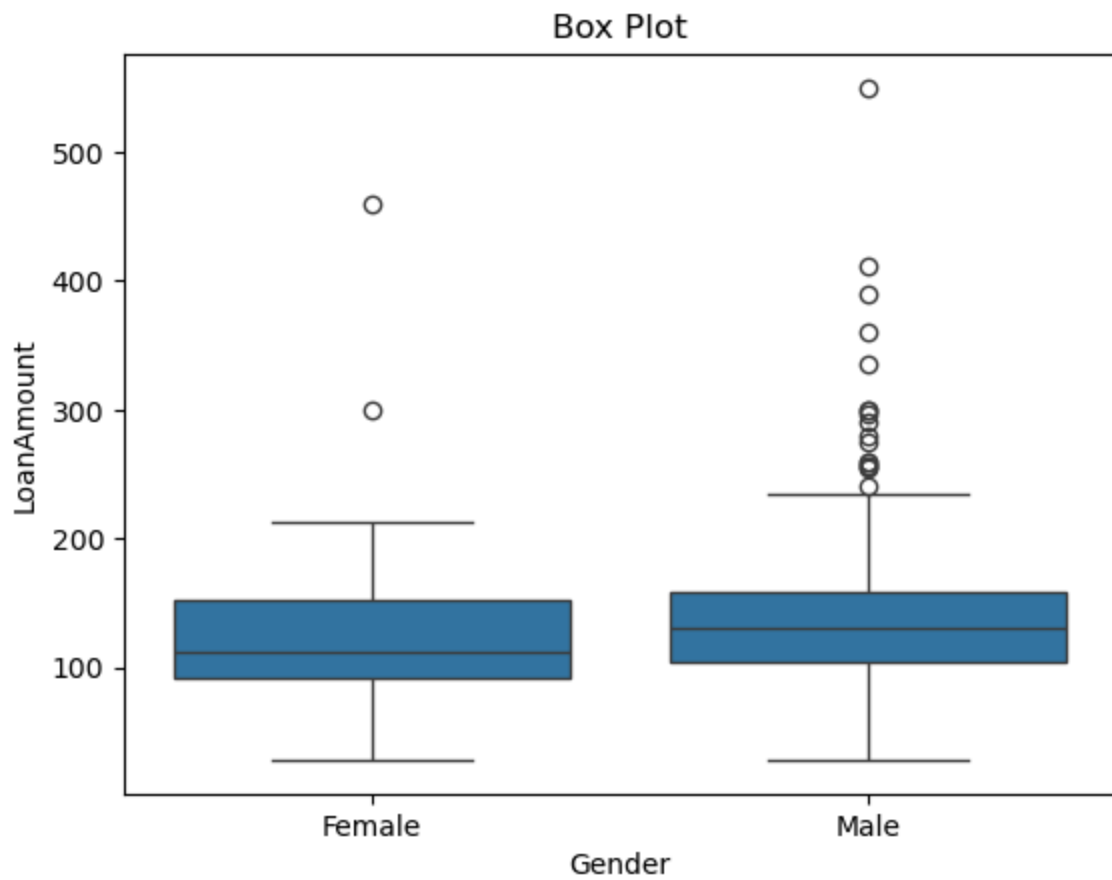
```
Out[58]: <function matplotlib.pyplot.show(close=None, block=None)>
```



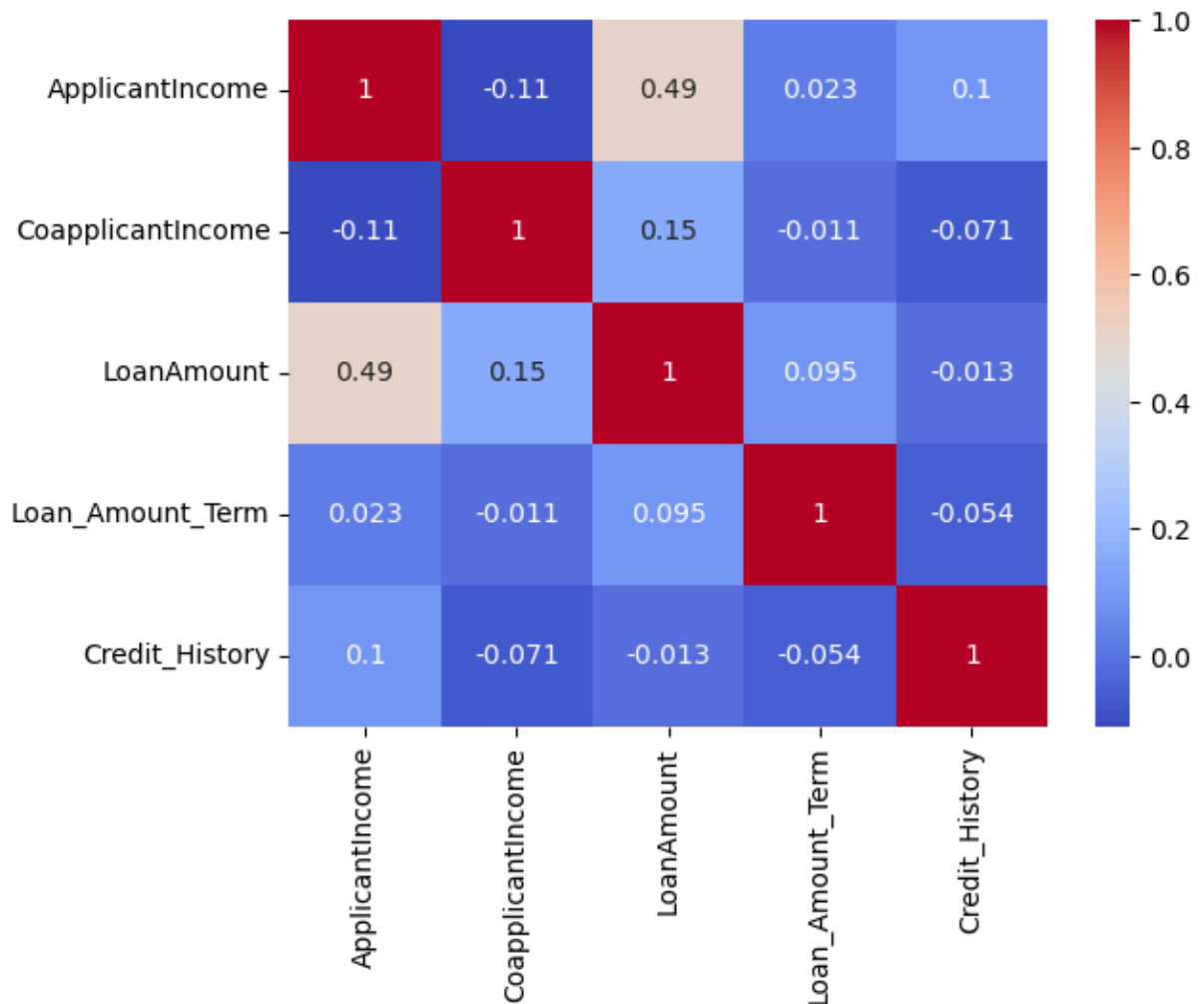
```
In [71]: sns.boxplot(data , x = 'Education' , y = 'LoanAmount')
plt.title("Box Plot")
plt.show()
```



```
In [73]: sns.boxplot(data , x = 'Gender' , y = 'LoanAmount')  
plt.title("Box Plot")  
plt.show()
```



```
In [75]: sns.heatmap(data.corr(numeric_only = True) , annot = True , cmap = 'coolwarm')  
plt.show()
```



```
In [77]: seg = data.groupby('Education')[['Loan_ID', 'LoanAmount']].sum().reset_index(seg)
```

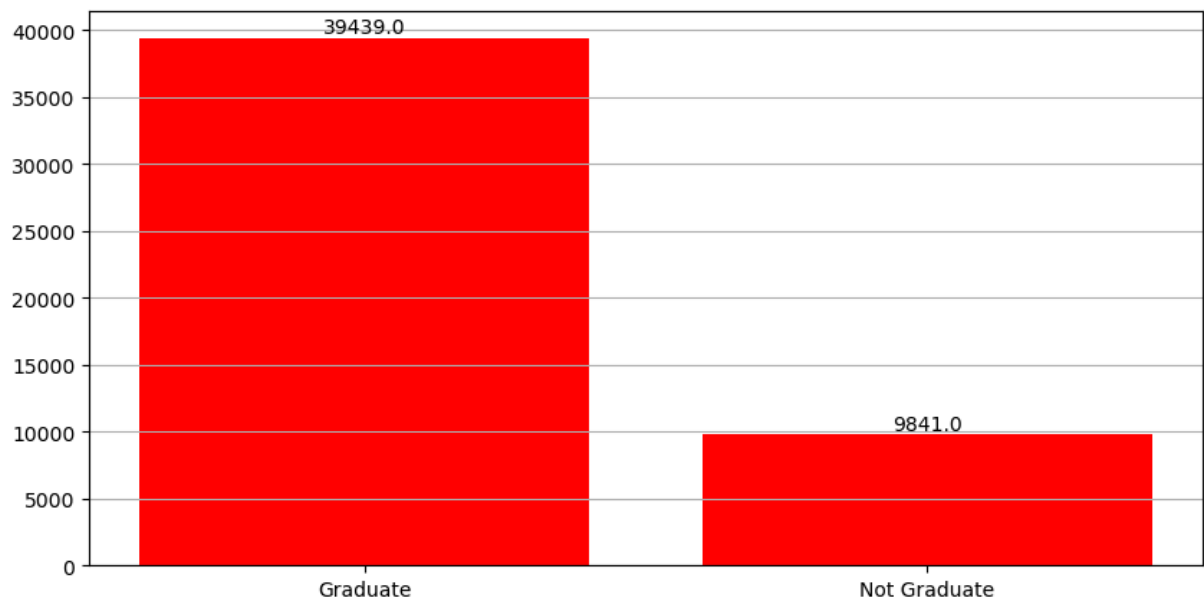
C:\Users\saransh\AppData\Local\Temp\ipykernel_2020\1665526032.py:1: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
seg = data.groupby('Education')[['Loan_ID', 'LoanAmount']].sum().reset_index()
```

```
Out[77]:
```

	Education	Loan_ID	LoanAmount
0	Graduate	LP001015LP001022LP001031LP001035LP001059LP0010...	39439.0
1	Not Graduate	LP001051LP001054LP001055LP001056LP001067LP0010...	9841.0

```
In [79]: plt.figure(figsize = (10,5))
plt.bar(seg.Education,seg.LoanAmount,color = 'red')
for i in range(len(seg)):
    plt.text(seg.Education[i],seg.LoanAmount[i],str(seg.LoanAmount[i]),ha='right')
plt.grid(axis = 'y')
plt.show()
```



In []:

In []: